UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

TEMA

GUIA MANUAL GIT

CURSO : **TALLER DE DESARROLLO DE APLICACIONES 1**

ESTUDIANTE **Cajahuaringa Camasca Rafael Max**

CICLO : **Iv**

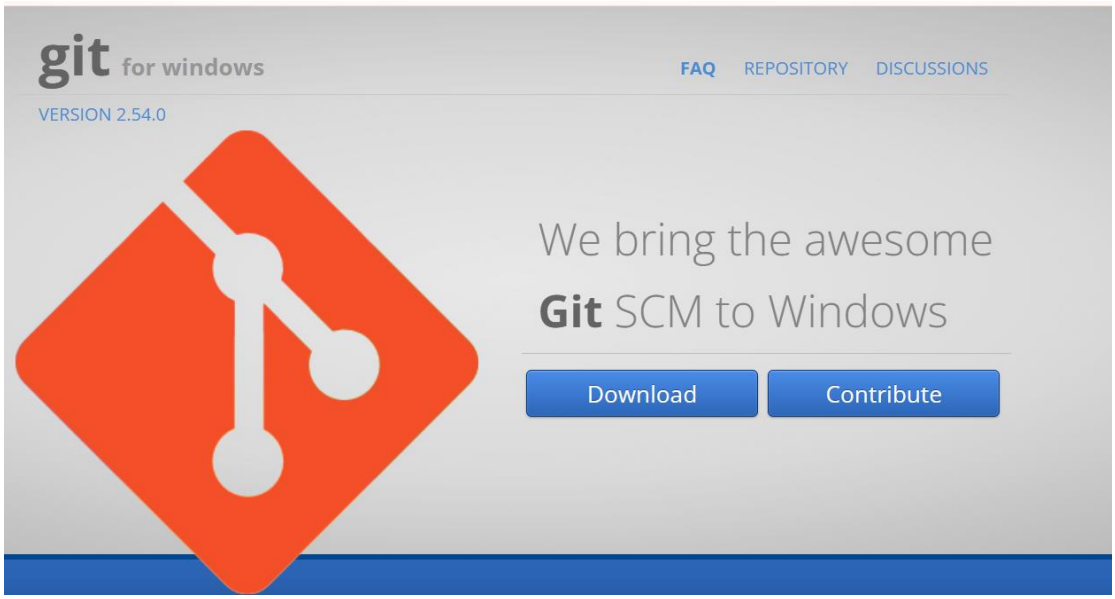
SECCION : **A1**

HUANCAYO – PERU

¿Qué es Git?

En términos sencillos, Git es un **sistema de control de versiones distribuido de código abierto**.

Los sistemas de control de versiones ayudan a cualquier equipo de software a gestionar cambios en el código fuente de un producto o servicio a lo largo del tiempo. Realiza un seguimiento de todas las modificaciones al código fuente en una base de datos. Si se ha cometido un error crítico en el código fuente, los desarrolladores de un equipo de software pueden retrotraer el código fuente a una versión antes de que se realizara el cambio erróneo. Como resultado, los sistemas de control de versiones protegen el código fuente de desastres, errores humanos y consecuencias no deseadas (cuando una corrección de errores rompe otra parte de la aplicación, por ejemplo).



¿En qué lenguaje está escrito Git?

Las herramientas que componen la distribución principal de Git están escritas en C, Shell, Perl y Tcl. Puedes encontrar el código fuente de Git en GitHub en git/git .

¿Por qué usar Git?

El control de versiones es fundamental; sin él, corres el riesgo de perder tu trabajo. Con Git, puedes crear un "commit" o punto de guardado con la frecuencia que desees. También puedes volver a commits anteriores. Esto te libera de la presión mientras trabajas. Confirma con frecuencia y al principio de la aplicación, y nunca tendrás esa sensación de angustia al sobrescribir o perder tus cambios.

Existen muchos sistemas de control de versiones, pero Git tiene algunas ventajas importantes.

Velocidad

Como ya mencionamos anteriormente, Git utiliza compresión SHA, lo que lo hace muy rápido.

Conflictos de fusión

Git puede gestionar conflictos de fusión, lo que significa que **varias personas pueden trabajar en el mismo archivo simultáneamente** . Esto abre un mundo de posibilidades para el desarrollo que no es posible con el control de versiones centralizado. Tienes acceso a todo el proyecto y, si trabajas en una rama, puedes hacer lo que necesites con la seguridad de que tus cambios están a salvo.

sucursales baratas

Hablando de ramas, Git ofrece mucha flexibilidad y oportunidades para la colaboración. **Al usar ramas, los desarrolladores pueden realizar cambios en un entorno seguro.**

En lugar de limitarse a implementar código que tenga un éxito 100% garantizado, los desarrolladores pueden implementar código que aún necesite ajustes. Luego, pueden subir ese código al repositorio remoto y obtener retroalimentación rápida mediante pruebas integradas o revisiones por pares.

Sin compartir el código a través de ramas, esto nunca sería posible.

Facilidad de reversión

Si cometes un error, ¡no pasa nada! Las confirmaciones son inmutables, lo que significa que no se pueden modificar. (Nota: Puedes cambiar el historial, pero se crearán nuevas confirmaciones de reemplazo en lugar de editar las existentes. ¡Más sobre esto más adelante!) Esto significa que si cometes un error, incluso en una rama importante, como `main`, no pasa nada . **Puedes revertir fácilmente ese cambio o volver al puntero de rama anterior, a la confirmación donde todo estaba bien.**

Los beneficios de esto son incalculables. No solo crea un entorno más seguro para el proyecto y el código, sino que fomenta un entorno de desarrollo donde los desarrolladores pueden ser más audaces, confiando en que Git los respalda.

¿Cómo utilizo Git?

Aprendiendo los conceptos básicos de Git

Si estás empezando con Git, un excelente lugar para aprender los comandos básicos es la Git Cheat Sheet . Está traducida a muchos idiomas, es de código abierto y forma parte del `github/training-kit`repositorio , además de ser un excelente punto de partida para los fundamentos de la línea de comandos.

Algunos de los comandos más importantes y utilizados que encontrarás allí son:

- `git clone [url]`: Clonar (descargar) un repositorio que ya existe en GitHub, incluyendo todos los archivos, ramas y confirmaciones.
- `git status`Siempre es una buena idea, este comando te muestra en qué rama te encuentras, qué archivos hay en el directorio de trabajo o de preparación, y cualquier otra información importante.
- `git branch`Esto muestra las ramas existentes en tu repositorio local. También puedes usarlo `git branch [branch-name]`para crear una rama desde tu ubicación actual o `git branch --all`para ver todas las ramas, tanto las locales en tu máquina como las ramas de seguimiento remotas almacenadas desde la última versión `git pull`o `git fetch`desde el repositorio remoto.
- `git checkout [branch-name]`: Cambia a la rama especificada y actualiza el directorio de trabajo.
- `git add [file]`: Crea una instantánea del archivo para prepararlo para el control de versiones y lo agrega al área de preparación.
- `git commit -m "descriptive message"`: Registra de forma permanente instantáneas de los archivos en el historial de versiones.
- `git pull`: Actualiza tu rama de trabajo local actual con todas las nuevas confirmaciones de la rama remota correspondiente en GitHub. `git pull`es una combinación de `git fetch`y `git merge`.
- `git push`: Sube todas las confirmaciones de la rama local al repositorio remoto.
- `git log`: Explore e inspeccione la evolución de los archivos del proyecto.
- `git remote -v`: Muestra los repositorios remotos asociados y su nombre almacenado, como `origin`.

Primeros pasos con el flujo de trabajo de Git

Dependiendo de tu sistema operativo, es posible que ya tengas Git instalado . Sin embargo, ¡empezar a usar Git implica más que solo tener el software! Para comenzar, es importante conocer los conceptos básicos de su funcionamiento. Puedes optar por trabajar directamente en una terminal, una aplicación como GitHub Desktop o a través de GitHub.com. (Nota: si bien puedes interactuar con Git a través de GitHub.com, tu experiencia podría ser limitada. Muchas herramientas locales te permiten acceder a las funcionalidades más utilizadas de Git, aunque solo la terminal te dará acceso a todas ellas) .

Crear una rama

La rama principal se suele llamar `main`. Queremos trabajar en otra rama para poder hacer una solicitud de extracción y realizar cambios de forma segura. Para empezar, crea una rama a partir de `main`. Nómbrala como quieras, pero recomendamos nombrar las ramas en función de la función o característica que será el foco de esta rama. Una persona puede tener varias ramas, y una rama puede tener varias personas colaborando en ella; las ramas tienen un propósito, no una persona. Dondequiera que estés actualmente (dondequiera que apunte HEAD, o cualquier rama en la que estés actualmente "extraído") será el padre de la rama que crees. Eso significa que puedes crear ramas a partir de otras ramas, etiquetas o cualquier commit. Pero, el flujo de trabajo más típico es crear una rama a partir de `main`, que representa el código de producción más reciente.

Realizar cambios (y confirmarlos)

Una vez que hayas creado una rama y movido el puntero HEAD a ella mediante el cambio a esa rama, ya puedes empezar a trabajar. Realiza los cambios en tu repositorio usando tu editor de texto o IDE favorito.

Envía tus cambios al control remoto

Hasta ahora, si has hecho una confirmación localmente, eres el único que puede verla. Para que otros vean tu trabajo y comiencen a colaborar, debes "enviar" tus cambios usando `git push`. Si estás enviando desde una rama que has creado localmente por primera vez, es posible que necesites darle a Git más información. `git push -u origin [branch-name]`le dice a Git que envíe la rama actual y cree una rama en el repositorio remoto que coincida con ella con el mismo nombre, y también, cree una relación con esa rama para que esa `git push`información sea suficiente en el futuro.

Por defecto, `git push`solo envía los cambios a la rama en la que te encuentras actualmente.